SCCM Task Sequence Troubleshooting & Status Message Analysis

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — SCCM Task Sequence Troubleshooting & Status Message Analysis 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — SCCM Task Sequence Troubleshooting & Status Message Analysis 5

What this solution is . . . 5

How a task sequence reports what happened . . . 5

Status messages that matter (IDs and severity) . . . 5

Where `smsts.log` lives — by phase . . . 6

The common failure classes . . . 6

Common exit codes → likely root cause . . . 7

The diagnose → resolve method . . . 7

The standard deployment model . . . 7

2. Requirements Guide — SCCM Task Sequence Troubleshooting & Status Message Analysis 8

2.1 Configuration Manager console access . . . 8

2.2 Read access to task sequences and deployments . . . 8

2.3 Configuration Manager PowerShell module . . . 8

2.4 Status message queries (Monitoring workspace) . . . 8

2.5 WMI / CIM access to the site's SMS provider . . . 8

2.6 Log access on the client and in WinPE . . . 9

2.7 Exit-code / status-message references . . . 9

2.8 A reproducible failure to pilot on . . . 9

Checklist . . . 9

3. Implementation Guide — SCCM Task Sequence Troubleshooting & Status Message Analysis 9

Step 1 — Assess: confirm the deployment state . . . 10

Step 2 — Configure: create the status message query . . . 10

Step 3 — Capture the exit code . . . 10

Step 4 — Open the phase-correct `smsts.log` . . . 10

Step 5 — Classify the failure . . . 11

Step 6 — Apply the class fix . . . 11

Step 7 — Re-run and confirm success . . . 11

Step 8 — Record the resolution . . . 12

Step 9 — Validate the runbook (pilot gate) . . . 12

Step 10 — Expand to the deployment team . . . 12
5. Testing Guide — SCCM Task Sequence Troubleshooting & Status Message Analysis 12
Test 1 — Deployment state is visible . . . 12
Test 2 — Status message query returns the execution messages . . . 12
Test 3 — Exit code captured and mapped . . . 12
Test 4 — Correct `smsts.log` opened and root cause found . . . 13
Test 5 — Class fix applied . . . 13
Test 6 — Re-run succeeds . . . 13
Test 7 — Failure summary report . . . 13
Pass/fail gate . . . 13
6. Troubleshooting Guide — SCCM Task Sequence Troubleshooting & Status Message Analysis 13
A. Status message query returns nothing . . . 13
B. WMI / SMS_StatusMessage access errors . . . 14
C. Can't find or open `smsts.log` . . . 14
D. ConfigurationManager PowerShell module won't import / cmdlets missing . . . 14
E. Exit code is `0x80004005` (unspecified) or blank . . . 15
F. Re-run doesn't pick up the fix . . . 15
G. Diagnostics quick reference . . . 15
H. When to escalate . . . 15
7. Best Practices — SCCM Task Sequence Troubleshooting & Status Message Analysis 15
8. Deliverables Summary — SCCM Task Sequence Troubleshooting & Status Message Analysis 16
Documentation . . . 16
PowerShell scripts . . . 17
What the buyer receives . . . 17
Outcome . . . 17
Appendix A — PowerShell Scripts 18

Executive Summary — SCCM Task Sequence Troubleshooting & Status Message Analysis

White-label · Infrastructure Solutions

When a Configuration Manager task sequence fails during OS deployment, the whole rollout stalls while engineers guess. The answer is buried in status messages, exit codes, and a sprawling `smsts.log` that changes location as the build moves from WinPE to the full OS. Guessing wastes hours per failure and re-images machines that only needed a one-line fix.

This solution is a structured, repeatable **runbook** for diagnosing task-sequence failures fast: read task-sequence states and status message IDs correctly, pull and parse `smsts.log` down to the failing step, map the exit code to a root cause, and resolve the failure by class — content and distribution point, drivers, network and PXE, application install, reboot, or task-sequence variables. It is written for a competent Configuration Manager administrator who needs a proven method instead of trial and error.

What you get

- A plain-language **Solution Overview** of how task sequences report state, what status message IDs and severities mean, where `smsts.log` lives per phase, and the diagnose-then-resolve model.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite (console access, status message queries, client and WinPE log access, exit-code references).
- An **Implementation Guide** — the ordered runbook: find the failing step via status messages, read the right `smsts.log`, map the exit code, fix by failure class, re-run, and record it.
- Five **PowerShell scripts** for task-sequence and status-message discovery, deployment-state validation, a failure-summary and exit-code report, safe remediation helpers, and a scope/collection targeting helper.
- **Testing** and **Troubleshooting** guides with explicit pass/fail gates.

What this delivers

A diagnostic capability your team keeps: every OSD failure is triaged the same way, to the exact step and exit code, so builds stop stalling and knowledge stops walking out the door. Pilot the runbook on real failures, prove it, then make it the standard.

The standard deployment model

Assess → Configure → Pilot → Validate → Expand.

Outcome

Task-sequence failures diagnosed to the exact step and exit code — status messages read correctly, `smsts.log` parsed, each failure class resolved from a proven runbook — validated on real failures before you scale it across the deployment team.

1. Solution Overview — SCCM Task Sequence Troubleshooting & Status Message Analysis

What this solution is

A structured method — a runbook plus tooling — for troubleshooting **Configuration Manager task sequences** that fail during OS deployment (OSD). Instead of re-imaging on a hunch, you read the signals Configuration Manager already produces: task-sequence deployment **states**, **status message IDs** and their severities, the `smsts.log` (which moves per phase), and the step **exit code**. Those signals point to one of a small set of **failure classes**, each with a known fix. The result is a repeatable diagnose-then-resolve loop any deployment engineer can run.

How a task sequence reports what happened

1. A task sequence is a deployment (advertisement) targeted at a **collection**. Each device's run

produces a **deployment status** — commonly `In Progress`, `Success`, `Failed`, `Error`, or `Unknown` — visible under Monitoring and per-deployment in the console.

1. As it runs, each step raises **status messages** to the management point. Status messages carry

a numeric **Message ID** and a **severity** (Informational, Warning, Error). Reading the right IDs tells you which step failed and why.

1. Every step also returns an **exit code**. `0` (and often `3010`, "success, reboot required") is

success; a non-zero code is the failing step's error — the single most important number to capture, because it usually maps directly to a root cause.

1. The full detail lives in `smsts.log`. Its **location changes by phase** (WinPE vs. full OS),

which is why so many people read the wrong log and stay stuck.

Status messages that matter (IDs and severity)

Task-sequence execution reports through the Task Sequence Engine. The IDs you will read most:

Message ID	Severity	Meaning
`11123`	Informational	The task sequence started (advertisement began execution).
`11124`	Error	The task sequence **failed**; the message includes the failing step and an exit/error code.
`11127`	Informational	The task sequence **completed successfully**.
`11170`	Error	A task-sequence action failed to run (engine-level step failure).
`11171`	Informational	A task-sequence action completed.

Message ID	Severity	Meaning
`10804` / `10805`	Info/Error	Content (package/application) download progress and failures on the client.

Content distribution and application deployment raise their own message classes (for example the `10000`-range for software distribution and app-install evaluation). Read the **Error**-severity message closest in time to the failure, then confirm the detail in `smsts.log`.

Where `smsts.log` lives — by phase

The log path depends on where the task sequence is when it fails:

Phase	`smsts.log` location
WinPE, before the disk is formatted	`x:\windows\temp\smstslog\smsts.log`
WinPE, after a disk is available	`x:\smstslog\smsts.log` or `<drive>:\_SMSTaskSequence\Logs\Smstslog\smsts.log`
Full OS, during the task sequence (client not yet installed)	`c:\_SMSTaskSequence\Logs\Smstslog\smsts.log`
Full OS, after the ConfigMgr client is installed	`c:\windows\ccm\logs\smsts.log`
After the task sequence ends (success or fail)	`c:\windows\ccm\logs\smsts.log` (final resting place on a client)

Read it with **CMTrace** (or OneTrace) so you can follow the timeline and filter on error lines. Search for the last `Failed`/`error`/`0x` line — the entries just above it name the step and reason.

The common failure classes

Almost every OSD failure falls into one of these, and each has a diagnostic tell and a fix:

• **Content / distribution point** — a package or application isn't on a DP the client can reach, or the DP is unhealthy. Tell: content-download errors (for example `0x80070002`, `0x87D00607`).

• **Drivers** — a driver package injected via Apply Driver Package or Auto Apply Drivers is wrong for the model, causing a bugcheck or a device that won't network. Tell: failure right after driver steps.

• **Network / PXE** — no PXE response, wrong boot image / architecture, or the client can't reach the MP/DP (missing NIC driver in the boot image, VLAN/DHCP option issues). Tell: fails in WinPE early.

• **Application / package install** — an app's installer returns a non-zero exit code or detection fails. Tell: exit code from the installer (for example `1603`, `1618`).

• **Reboot / continuation** — the task sequence doesn't resume after a restart (boot order, BitLocker, or the engine losing its state). Tell: log stops at a Restart step.

• **Task-sequence variables / logic** — a condition, collection variable, or missing variable sends

the sequence down the wrong path or skips a required step. Tell: a step is skipped or runs wrongly.

Common exit codes → likely root cause

Exit code (dec / hex)	Class	Likely cause
`0` / `0x0`	—	Success.
`3010` / `0xBC2`	—	Success, **reboot required** (not a failure).
`2` / `0x80070002`	Content	File not found — content missing on the DP, or a wrong path/name.
`-2147024891` / `0x80070005`	Permissions	Access denied — network access account, share ACLs, or rights.
`1603` / `0x643`	App install	Fatal error during MSI installation (per the app's own log).
`1618` / `0x652`	App install	Another install is in progress — serialize installs / add a retry.
`1620` / `0x654`	App install	Installer package could not be opened (corrupt/incomplete content).
`-2147467259` / `0x80004005`	Generic	Unspecified failure — read `smsts.log` for the real inner error.
`2278886630` / `0x87D00607`	Content	Content download failed / not found on any DP.
`2278887145` / `0x87D00269`	Client/policy	Software distribution/policy error — check MP/policy retrieval.

Always map the code, then confirm the cause in `smsts.log` — the exit code narrows the search; the log confirms it.

The diagnose → resolve method

> ***1. Identify the failed run*** *(deployment state + status message ID* `11124/11170`*).* ***2. Get the exit code*** *from the status message.* ***3. Open the right*** `smsts.log` ***for the phase*** *and read up from the last error.* ***4. Classify*** *the failure (content / driver / network / app / reboot / variable).* ***5. Apply the class fix.*** ***6. Re-run and confirm*** *(*`11127` */ exit* `0`*).* ***7. Record*** *the ID, code, cause, and fix so the next person is faster.*

The standard deployment model

> ***Assess*** *the failing OSD →* ***Configure*** *access to status message queries and logs →* ***Pilot*** *the runbook on real failures →* ***Validate*** *the diagnoses and fixes →* ***Expand*** *it to the whole deployment team. repeatable pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***

2. Requirements Guide — SCCM Task Sequence Troubleshooting & Status Message Analysis

Verify **every** requirement before you rely on the runbook. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-infrastructure.md`.

2.1 Configuration Manager console access

Description: The Configuration Manager console connected to your primary site, on a current build. **Why:** You diagnose deployment state, status messages, and task-sequence deployments from the console. **Verify:** Launch the console; it connects and shows **Software Library**, **Monitoring**, and **Assets and Compliance**. **Implement:** Install the console from `<CMinstall>\tools\ConsoleSetup` (or Software Center) and point it at the site server.

2.2 Read access to task sequences and deployments

Description: An RBAC role that grants read on task sequences, deployments, and status messages (for example **Read-only Analyst**, or **OS Deployment Manager** for changes). **Why:** You must see the task-sequence deployments, their targeted collections, and the status messages they raise. **Verify:** Console → **Software Library** → **Operating Systems** → **Task Sequences** lists sequences; **Monitoring** → **Deployments** shows OSD deployments. **Implement:** Administration → **Security** → **Administrative Users**; assign a role scoped to the relevant collections.

2.3 Configuration Manager PowerShell module

Description: The ConfigurationManager PowerShell module (ships with the console). **Why:** The discovery, validation, report, and targeting scripts use `Get-CM*` cmdlets against the site. **Verify:** In the console, use the top-left dropdown → **Connect via Windows PowerShell ISE**, or run `Import-Module "$($ENV:SMS_ADMIN_UI_PATH)\..\ConfigurationManager.psd1"` — no error. **Implement:** Install the console (2.1); the module is at `<console>\bin\ConfigurationManager.psd1`. Run scripts from a session with the site drive (`PS XYZ:\>`).

2.4 Status message queries (Monitoring workspace)

Description: Access to **Monitoring** → **System Status** → **Status Message Queries**, plus the ability to save custom queries. **Why:** Status message queries are how you pull the task-sequence execution messages (IDs `11124`, `11170`, `11127`) for a device or collection. **Verify:** Console → **Monitoring** → **System Status** → **Status Message Queries** opens and you can run the built-in queries. **Implement:** Granted by the RBAC role in 2.2; confirm the operator can create and save a query.

2.5 WMI / CIM access to the site's SMS provider

Description: Read access to the `root\SMS\site_<code>` WMI namespace (the SMS provider) from the operator's workstation. **Why:** The report script queries the `SMS_StatusMessage` class (and related classes) to summarize failures and exit codes. **Verify:** Run `Get-CimInstance -Namespace "root\SMS\site_<code>" -ClassName SMS_StatusMessage -ComputerName`

`<SMSProvider> -Filter "MessageID=11124"` — returns rows or an empty set, not an access error. **Implement:** The RBAC role in 2.2 grants provider access; ensure the account can reach the SMS Provider server over WMI (firewall/DCOM).

2.6 Log access on the client and in WinPE

Description: The ability to read `smsts.log` at every phase path (WinPE and full OS) and a log viewer. **Why:** Status messages point at the step; `smsts.log` confirms the exact reason. You must be able to open the phase-correct log. **Verify:** On a failed client, `c:\windows\ccm\logs\smsts.log` exists (or the `_SMSTaskSequence` path); in WinPE, press **F8** at a command prompt and browse `x:\windows\temp\smstslog`. **Implement:** Enable command-line support (F8) on the boot image (**Software Library** → **Boot Images** → **Properties** → **Customization** → **Enable command support (testing)**); install **CMTrace/OneTrace** for reading logs.

2.7 Exit-code / status-message references

Description: A reference for Windows/MSI exit codes and Configuration Manager status message IDs. **Why:** Mapping a captured exit code to a root cause is faster with a reference than by memory. **Verify:** You have the exit-code and status-ID tables (this solution's Overview, §1) to hand. **Implement:** Use the tables in `01-solution-overview.md`; the report script also prints the codes it finds.

2.8 A reproducible failure to pilot on

Description: At least one real, current task-sequence failure (a device or collection) to run the runbook against. **Why:** The runbook is validated by diagnosing actual failures — not by reading it. **Verify:** Console → **Monitoring** → **Deployments** shows an OSD deployment with a `Failed`/`Error` device. **Implement:** Use a currently failing build; if none exists, re-run a known-bad task sequence on a test device to generate one.

Checklist

- Configuration Manager console connects to the site
- RBAC read on task sequences, deployments, and status messages
- ConfigurationManager PowerShell module imports
- Status Message Queries accessible in Monitoring
- WMI/CIM access to `root\SMS\site_<code>` (SMS_StatusMessage)
- `smsts.log` readable at WinPE + full-OS paths; F8 enabled; CMTrace available
- Exit-code / status-ID reference on hand
- A real failure identified to pilot the runbook on

3. Implementation Guide — SCCM Task Sequence Troubleshooting & Status Message Analysis

Follow these steps **in order**. This is the runbook: it takes a failed task-sequence deployment and drives it to a root cause and a fix. Console work is in the **Configuration Manager console**; scripts run from a **ConfigurationManager PowerShell** session (site drive `PS XYZ:\>`).

> ***Model:*** *Assess the failure → configure the queries and log access → pilot the runbook on a real failure → validate the diagnosis and fix → expand it to the deployment team. Name every artifact you create with* `<Purpose> – <Scope> – <Stage>` *and reuse the names.*

Step 1 — Assess: confirm the deployment state

1. Console → **Monitoring** → **Deployments**. Find the task-sequence deployment; note **Success**, **In Progress**, **Error**, and **Unknown** counts.

1. Open the deployment → **View Status** → the **Error** and **In Progress** tabs list the failing devices. Record the device name(s) and the last state.

1. Alternatively run `SCCMTS-Discovery.ps1` to list task-sequence deployments and their recent per-device states from the site.

Step 2 — Configure: create the status message query

1. Console → **Monitoring** → **System Status** → **Status Message Queries** → **Create Status Message Query**.

2. **Name:** `TS Execution Errors – OSD – Triage` (reuse this exact name). **Edit Query Statement →

Show Query Language **and target the task-sequence engine messages for the device/time window: filter** Message ID** in (`11123`, `11124`, `11127`, `11170`, `11171`) and set the machine name / date prompt.

1. **OK** → **Run**. When prompted, enter the failing device and a time range covering the run.

2. Read the results newest-first: `11124` (or `11170`) is the failure and carries the **failing step name **and an** exit/error code**. `SCCMTS-Discovery.ps1` returns the same messages if you prefer script output.

Step 3 — Capture the exit code

1. From the `11124`/`11170` message, record the **step name** and the **exit code** (decimal and hex). This is the key that narrows the search.

1. Map it against the exit-code table in `01-solution-overview.md` (or run `SCCMTS-Report.ps1`, which summarizes failures and breaks them down by exit code across the collection).

Step 4 — Open the phase-correct `smsts.log`

1. Determine the phase from where the sequence failed (WinPE vs. full OS) — the status message time and step name tell you.

1. Open the matching log with **CMTrace**:

• WinPE (pre-format): `x:\windows\temp\smstslog\smsts.log` (press **F8** in WinPE to get a prompt).

• WinPE (disk available): `<drive>:\_SMSTaskSequence\Logs\Smstslog\smsts.log`.

• Full OS, client not yet installed: `c:\_SMSTaskSequence\Logs\Smstslog\smsts.log`.

• Full OS, client installed / after completion: `c:\windows\ccm\logs\smsts.log`.

1. Search up from the last `Failed`/`error`/`0x...` line. The entries just above it name the action, the command, and the inner reason. That is your root cause.

Step 5 — Classify the failure

Using the exit code (Step 3) and the log detail (Step 4), assign the failure to a class: `Content/DP`, `Drivers`, `Network/PXE`, `Application install`, `Reboot/continuation`, or `Task-sequence variables/logic` (see §1 for each class's tell).

Step 6 — Apply the class fix

• **Content / DP:** Confirm the package/app is distributed to a DP the client reaches (**Monitoring** → **Distribution Status** → **Content Status**); redistribute if `Failed`; validate the DP. Check the boundary/boundary-group so the client selects a reachable DP.

• **Drivers:** Replace the wrong driver package; scope Apply Driver Package by model with a WMI condition, or fix Auto Apply Drivers categories. Re-distribute the driver package.

• **Network / PXE:** Add the correct NIC driver to the **boot image** and update DPs; verify the boot image architecture matches the hardware; confirm DHCP/PXE (IP Helpers or options) and PXE responder.

• **Application install:** Read the app's own installer log for the exit code (for example `1603`, `1618`); fix the install command/switches or detection; add a serialize/retry for `1618`.

• **Reboot / continuation:** Fix boot order so the OS disk boots; suspend BitLocker during the build; ensure the Restart Computer step targets the right OS/WinPE.

• **Variables / logic:** Correct the step **condition**, collection/task-sequence variable, or the missing variable that steered the run; re-check with the variable listing.

Step 7 — Re-run and confirm success

1. Re-deploy / re-run the task sequence on the piloted device (or clear the last run so it re-evaluates — see `SCCMTS-Remediation.ps1`, which is `-WhatIf` by default).

1. Confirm success: status message `11127` and step exit code `0` (or `3010`), and the deployment shows **Success**. Run `SCCMTS-Validation.ps1` to confirm the deployment state for the device/collection.

Step 8 — Record the resolution

1. Capture the status ID, exit code, class, root cause, and fix in your runbook log (name it `OSD Failure Log – <Site> – Runbook`). This is what makes the next diagnosis faster.

Step 9 — Validate the runbook (pilot gate)

1. Run the full loop (Steps 1–8) on **at least two** real, different failures. Confirm each was diagnosed to the correct step and exit code and resolved. Complete the **Testing Guide**.

Step 10 — Expand to the deployment team

1. Save the status message query, the scripts, and the recorded resolutions where the team works.
2. Make this the standard triage path for OSD failures; re-validate periodically as boot images, drivers, and apps change.

Order of operations

`Assess state → Status message query → Capture exit code → Read phase-correct smsts.log → Classify → Fix by class → Re-run + confirm 11127/0 → Record → Validate on real failures → Expand`

5. Testing Guide — SCCM Task Sequence Troubleshooting & Status Message Analysis

Run after you build the query and pilot the runbook (Implementation Steps 2 and 9). These tests prove the runbook actually diagnoses and resolves real failures — not that the doc reads well.

Test 1 — Deployment state is visible

Do: Console → **Monitoring** → **Deployments** → the OSD deployment → **View Status**, or run `SCCMTS-Discovery.ps1`. **Expected:** The failing device(s) and their last state (`Failed`/`Error`) are listed.

Test 2 — Status message query returns the execution messages

Do: Run the `TS Execution Errors – OSD – Triage` query for the failing device and time window. **Expected:** Messages `11123`/`11170`/`11124` appear; the `11124`/`11170` message names the failing step and carries an exit/error code.

Test 3 — Exit code captured and mapped

Do: Record the exit code from Test 2; check it against the table in `01-solution-overview.md` (or run `SCCMTS-Report.ps1`). **Expected:** The code maps to a failure class (content / driver / network / app / reboot / variable).

Test 4 — Correct `smsts.log` opened and root cause found

Do: Open the phase-correct `smsts.log` (WinPE vs. full OS path) in CMTrace; read up from the last error line. **Expected:** The log confirms the step, the command, and the inner reason consistent with the exit code.

Test 5 — Class fix applied

Do: Apply the fix for the identified class (Implementation Step 6). **Expected:** The blocking condition is corrected (content on a reachable DP, right driver/boot NIC, fixed installer switch/detection, boot order/BitLocker, or corrected variable/condition).

Test 6 — Re-run succeeds

Do: Re-run the task sequence on the piloted device; run `SCCMTS-Validation.ps1`. **Expected:** Status message `11127` and step exit `0` (or `3010`); the deployment shows **Success** for the device.

Test 7 — Failure summary report

Do: Run `SCCMTS-Report.ps1` for the collection. **Expected:** A clean failure summary with an exit-code breakdown and the affected devices.

Pass/fail gate

Gate	Requirement
Diagnosis pass	Tests 1–4 pass: state seen, query returns the message, exit code mapped, log confirms the cause
Resolution pass	Tests 5–6 pass: the class fix is applied and the re-run reports `11127` / exit `0`
Runbook pass	The full loop is validated on **two** distinct real failures (Implementation Step 9)

6. Troubleshooting Guide — SCCM Task Sequence Troubleshooting & Status Message Analysis

This is the formal troubleshooting section for the **runbook and its tooling** — what to do when the diagnostic process itself misbehaves (queries return nothing, WMI is denied, the log is in a place you didn't expect, the module won't load). For diagnosing the *task-sequence failure*, follow the Implementation runbook (§3).

A. Status message query returns nothing

Symptom	Cause	Remediation
Query runs but no rows	Wrong device name / time window, or messages haven't summarized yet	Widen the time range; use the exact NetBIOS name; wait for status summarization (or refresh the summarizer)
No `11124`/`11170` at all	The run never reported (client couldn't reach the MP), or wrong Message IDs	Confirm the client reached the MP; verify the query filters IDs `11123,11124,11127,11170,11171`
RBAC hides the messages	Operator's role isn't scoped to the collection	Scope the RBAC role (Req 2.2) to the target collection

B. WMI / SMS_StatusMessage access errors

• `Access denied` querying `root\SMS\site_<code>` → the account lacks provider access or DCOM/WMI is

blocked. Grant the RBAC role (Req 2.2/2.5) and open WMI to the SMS Provider server.

• Empty `SMS_StatusMessage` results → confirm you're querying the **SMS Provider** server and the correct three-character **site code**; check the `MessageID` filter.

C. Can't find or open `smsts.log`

Symptom	Cause	Remediation
Log not at `c:\windows\ccm\logs`	Failure happened in WinPE or before client install	Use the phase path: `x:\windows\temp\smstslog` (WinPE) or `c:\_SMSTaskSequence\Logs\Smstslog`
WinPE reboots before you can read it	No command support on the boot image	Enable **Enable command support (F8)** on the boot image; capture the log before the reboot, or add an SMSTSLogPath step
Log is unreadable / truncated	Opened in Notepad; or rotated	Use CMTrace/OneTrace; check for `smsts-*.log` rollover files in the same folder

D. ConfigurationManager PowerShell module won't import / cmdlets missing

• `Import-Module` fails → import from the console path:

`Import-Module "$($ENV:SMS_ADMIN_UI_PATH)\..\ConfigurationManager.psd1"`, then `cd XYZ:` (your site drive) before running `Get-CM*` cmdlets.

• `Get-CMTaskSequenceDeployment` returns nothing → confirm you're on the site drive and the RBAC scope

includes the deployments; check the exact task-sequence/deployment name.

E. Exit code is `0x80004005` (unspecified) or blank

- `0x80004005` is generic — the real error is the **inner** failure in `smsts.log`. Read up from the last error line for the actual command/return code; don't act on `0x80004005` alone.
- Blank/missing exit code → the step may have failed before returning (timeout, engine crash); correlate with the smsts.log timestamp of the `11170` message.

F. Re-run doesn't pick up the fix

- Cached policy / last-run state → force a machine policy retrieval, or clear the prior run state and re-deploy. `SCCMTS-Remediation.ps1` (default `-WhatIf`) shows the retry and ccmcache-clear actions; re-run with `-WhatIf:$false` to execute.
- Content still stale on the client → clear ccmcache so the client re-downloads corrected content.

G. Diagnostics quick reference

```
# On the client — find the active/most recent smsts.log
Get-ChildItem 'C:\Windows\CCM\Logs\smsts*.log','C:\_SMSTaskSequence\Logs\Smstslog\smsts*.log' -EA Silently
    Sort-Object LastWriteTime -Descending | Select-Object FullName,LastWriteTime

# From an admin workstation — task-sequence execution failure messages for a device
Get-CimInstance -Namespace 'root\SMS\site_<code>' -ComputerName '<SMSProvider>' `
    -ClassName SMS_StatusMessage -Filter 'MessageID=11124' -EA SilentlyContinue |
    Select-Object -First 20 MachineName,Time,MessageID,SiteCode
```

Open logs with CMTrace (`cmtrace.exe smsts.log`). The last `0x...` line is the failure; read upward.

H. When to escalate

If the status message query returns the failure, the exit code is mapped, the phase-correct `smsts.log` is read, the class fix is applied, and the re-run still fails with the same code — collect `smsts.log` (all rollover files) plus the client's `CAS.log`/`ContentTransferManager.log` (content failures) and open a Microsoft support case with the step name and exit code.

7. Best Practices — SCCM Task Sequence Troubleshooting & Status Message Analysis

- **Always read the phase-correct log.** Most stalled troubleshooting is someone reading the wrong `smsts.log`. Confirm the phase (WinPE vs. full OS) first, then open the matching path.
- **Capture the exit code before anything else.** The non-zero exit code from the `11124`/`11170` message is the fastest key to the root cause — record it, then confirm in the log.
- **Enable command support (F8) on boot images** in your lab/pilot so you can browse `x:\windows\temp\smstslog` live during a WinPE failure. Disable it on production boot images.

• **Use CMTrace / OneTrace, never Notepad.** Timeline view and error highlighting turn a sprawling log into a readable failure chain.

• **Save a reusable status message query.** Keep `TS Execution Errors – OSD – Triage` so every engineer triages the same way instead of rebuilding the query each time.

• **Check content status early.** A large share of OSD failures are content on an unreachable or unhealthy DP — verify **Content Status** and boundary groups before deeper digging.

• **Keep the boot image current.** Add the right NIC/storage drivers and update DPs after any hardware refresh; a missing boot NIC driver is the classic PXE/network failure.

• **Fix the class, not the symptom.** Classify (content / driver / network / app / reboot / variable) and apply the class fix — re-imaging without a diagnosis just repeats the failure.

• **Record every resolution.** A short log of ID → exit code → cause → fix compounds: the second occurrence of a failure should be minutes, not hours.

• **Pilot the runbook on real failures.** Validate it on at least two distinct failures before making it the team standard — assign once, repeat everywhere.

8. Deliverables Summary — SCCM Task Sequence Troubleshooting & Status Message Analysis

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	How task sequences report state, status message IDs/severities, `smsts.log` per phase, exit codes, failure classes, the diagnose-then-resolve model
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Infrastructure)	Category-wide prerequisites, published once
3. Implementation Guide	The ordered runbook: state → query → exit code → log → classify → fix → confirm → record → validate → expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Troubleshooting the runbook itself (queries, WMI, logs, module)
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`SCCMTS-Discovery.ps1`	Lists task sequences + their deployments, recent per-device deployment states, and task-sequence status messages (read-only)
`SCCMTS-Validation.ps1`	Validates the deployment state for a device or collection (read-only)
`SCCMTS-Report.ps1`	Failure summary with an exit-code breakdown across a collection (read-only)
`SCCMTS-Remediation.ps1`	Safe remediation helpers (`-WhatIf` by default): retry a deployment and clear ccmcache
`SCCMTS-Set-ScopeTargeting.ps1`	Collection / scope targeting helper for piloting the runbook

What the buyer receives

- This runbook as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the repeatable model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Task-sequence failures diagnosed to the exact step and exit code — status messages read correctly, `smsts.log` parsed to the failing step, each failure class resolved from a proven runbook — so OS deployments stop stalling, validated on real failures before you scale it out.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SCCMTS-Discovery.ps1

Purpose: Read-only triage inventory: lists task sequences and their deployments (Get-CM* cmdlets), the most recent per-device deployment states for a task sequence, and the task-sequence engine status messages (IDs 11123/11124/11127/11170/11171) from the SMS provider via CIM. This is Step 1-2 of the runbook — see where the failure is and pull the failing message. HOW TO RUN From a ConfigurationManager PowerShell session (site drive PS XYZ:\>): .\SCCMTS-Discovery.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com .\SCCMTS-Discovery.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -DeviceName PC123 -Hours 24 Requires the ConfigurationManager module (console) and RBAC read on TS/deployments/messages. EXPECTED RESULTS A list of task sequences + deployments, and (if -DeviceName given) the recent 11124/11170 failure messages for that device, newest first, with the exit/error code text. .NOTES White-label — Infrastructure Solutions. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$SiteCode, [Parameter(Mandatory)][string]$SmsProvider, [string]$DeviceName, [int]$Hours = 24)

How to run: From a ConfigurationManager PowerShell session (site drive PS XYZ:\>): .\SCCMTS-Discovery.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com .\SCCMTS-Discovery.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -DeviceName PC123 -Hours 24 Requires the ConfigurationManager module (console) and RBAC read on TS/deployments/messages. EXPECTED RESULTS A list of task sequences + deployments, and (if -DeviceName given) the recent 11124/11170 failure messages for that device, newest first, with the exit/error code text. .NOTES White-label — Infrastructure Solutions. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$SiteCode, [Parameter(Mandatory)][string]$SmsProvider, [string]$DeviceName, [int]$Hours = 24)

SCCMTS-Remediation.ps1

Purpose: After a fix (runbook Step 6), help the device re-evaluate cleanly: - Retry: trigger a machine policy retrieval + evaluation cycle on the target device so the re-deployed task sequence is re-offered (via CIM SMS Client SDK / trigger schedule). - Clear ccmcache: remove stale/corrupt cached content so corrected content re-downloads. This script is SAFE BY DEFAULT: with -WhatIf (the default) it only prints what it would do. Re-run with -WhatIf:$false to actually perform the actions. HOW TO RUN Preview (default, no changes): .\SCCMTS-Remediation.ps1 -DeviceName PC123 -Retry .\SCCMTS-Remediation.ps1 -DeviceName PC123 -ClearCache Execute: .\SCCMTS-Remediation.ps1 -DeviceName PC123 -Retry -ClearCache -WhatIf:$false Requires remote CIM/WMI access to the target client (admin rights). EXPECTED RESULTS With -WhatIf: a list of the exact actions. Without: policy cycles triggered and/or ccmcache cleared; then re-run SCCMTS-Validation.ps1 after a policy cycle to confirm 11127 / exit 0. .NOTES White-label — Infrastructure Solutions. Changes are gated behind -WhatIf. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$DeviceName, [switch]$Retry, [switch]$ClearCache)

How to run: Preview (default, no changes): .\SCCMTS-Remediation.ps1 -DeviceName PC123 -Retry .\SCCMTS-Remediation.ps1 -DeviceName PC123 -ClearCache Execute: .\SCCMTS-Remediation.ps1 -DeviceName PC123 -Retry -ClearCache -WhatIf:$false Requires remote CIM/WMI access to the target client (admin rights). EXPECTED RESULTS With -WhatIf: a list of the exact actions. Without: policy cycles

triggered and/or ccmcache cleared; then re-run SCCMTS-Validation.ps1 after a policy cycle to confirm 11127 / exit 0. .NOTES White-label — Infrastructure Solutions. Changes are gated behind -WhatIf. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$DeviceName, [switch]$Retry, [switch]$ClearCache)

SCCMTS-Report.ps1

Purpose: Read-only reporting: pulls task-sequence failure status messages (IDs 11124/11170) for a collection or the whole site over a window, extracts the exit/error code from each message, and prints a breakdown by exit code plus the affected devices. This is the fleet view of the runbook — which failures are common and which exit codes to chase first. HOW TO RUN From a ConfigurationManager PowerShell session: .\SCCMTS-Report.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -Days 7 .\SCCMTS-Report.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -Days 7 -CsvPath C:\Temp\ts.csv Requires the ConfigurationManager module and RBAC read on status messages. EXPECTED RESULTS A count of failures, a table grouped by exit code (with the likely class), a per-device list, and an optional CSV export. Read-only against the SMS provider. .NOTES White-label — Infrastructure Solutions. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$SiteCode, [Parameter(Mandatory)][string]$SmsProvider, [int]$Days = 7, [string]$CollectionID, [string]$CsvPath)

How to run: From a ConfigurationManager PowerShell session: .\SCCMTS-Report.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -Days 7 .\SCCMTS-Report.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -Days 7 -CsvPath C:\Temp\ts.csv Requires the ConfigurationManager module and RBAC read on status messages. EXPECTED RESULTS A count of failures, a table grouped by exit code (with the likely class), a per-device list, and an optional CSV export. Read-only against the SMS provider. .NOTES White-label — Infrastructure Solutions. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$SiteCode, [Parameter(Mandatory)][string]$SmsProvider, [int]$Days = 7, [string]$CollectionID, [string]$CsvPath)

SCCMTS-Set-ScopeTargeting.ps1

Purpose: Prepare a small, controlled scope to pilot the troubleshooting runbook on real failures without touching production breadth: - Ensure a pilot device collection exists (create it if missing). - Add the specific failing device(s) to that pilot collection. Use it so re-runs and remediation stay confined to the piloted devices (runbook Steps 9-10). SAFE BY DEFAULT: -WhatIf is on unless you pass -WhatIf:$false. HOW TO RUN From a ConfigurationManager PowerShell session (site drive PS XYZ:\>): Preview: .\SCCMTS-Set-ScopeTargeting.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com ` -CollectionName "OSD - Runbook - Pilot" -DeviceName PC123 Execute: add -WhatIf:$false Requires the ConfigurationManager module and RBAC rights to create/modify collections. EXPECTED RESULTS With -WhatIf: the collection and membership changes it would make. Without: the pilot collection exists and the named device(s) are direct members, ready to pilot the runbook. .NOTES White-label — Infrastructure Solutions. Changes are gated behind -WhatIf. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$SiteCode, [Parameter(Mandatory)][string]$SmsProvider, [Parameter(Mandatory)][string]$CollectionName, [string[]]$DeviceName, [string]$LimitingCollectionName = 'All Systems')

How to run: From a ConfigurationManager PowerShell session (site drive PS XYZ:\>): Preview: .\SCCMTS-Set-ScopeTargeting.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com ` -CollectionName "OSD - Runbook - Pilot" -DeviceName PC123 Execute: add -WhatIf:$false Requires the ConfigurationManager module and RBAC rights to create/modify collections. EXPECTED RESULTS With

-WhatIf: the collection and membership changes it would make. Without: the pilot collection exists and the named device(s) are direct members, ready to pilot the runbook. .NOTES White-label — Infrastructure Solutions. Changes are gated behind -WhatIf. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$SiteCode, [Parameter(Mandatory)][string]$SmsProvider, [Parameter(Mandatory)][string]$CollectionName, [string[]]$DeviceName, [string]$LimitingCollectionName = 'All Systems')

SCCMTS-Validation.ps1

Purpose: Read-only check of where a task-sequence deployment currently stands. For a device it reports the last deployment/execution status; for a collection it summarizes Success / In Progress / Error / Failed counts. Use after a fix + re-run to confirm success (status 11127 / exit 0). HOW TO RUN From a ConfigurationManager PowerShell session (site drive PS XYZ:\>): .\SCCMTS-Validation.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -DeviceName PC123 .\SCCMTS-Validation.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -CollectionName "OSD - Pilot" Requires the ConfigurationManager module and RBAC read on deployments/status messages. EXPECTED RESULTS "DEPLOYMENT SUCCEEDED" (exit 0) when the device's latest result is success (11127), or a collection status breakdown; "NOT SUCCEEDED" (exit 1) if a failure is still present. .NOTES White-label — Infrastructure Solutions. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$SiteCode, [Parameter(Mandatory)][string]$SmsProvider, [string]$DeviceName, [string]$CollectionName)

How to run: From a ConfigurationManager PowerShell session (site drive PS XYZ:\>): .\SCCMTS-Validation.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -DeviceName PC123 .\SCCMTS-Validation.ps1 -SiteCode ABC -SmsProvider cm01.contoso.com -CollectionName "OSD - Pilot" Requires the ConfigurationManager module and RBAC read on deployments/status messages. EXPECTED RESULTS "DEPLOYMENT SUCCEEDED" (exit 0) when the device's latest result is success (11127), or a collection status breakdown; "NOT SUCCEEDED" (exit 1) if a failure is still present. .NOTES White-label — Infrastructure Solutions. Read-only. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$SiteCode, [Parameter(Mandatory)][string]$SmsProvider, [string]$DeviceName, [string]$CollectionName)